# Relatório de Auditoria de Segurança Vigora

Aplicativo de monitoramento de saúde para idosos
React Native · Expo · tRPC · Drizzle/MySQL

| | |
|---|---|
| **Data** | 4 de setembro de 2026 |
| **Escopo** | Backend completo (server/, drizzle/, shared/), aplicativo (app/, lib/, components/, hooks/, context/), configuração de build e deploy (app.config.ts, eas.json, .github/workflows/) e histórico do Git. |
| **Revisão** | branch `fix/launch-prep`, commit `fb7f379` |
| **Cobertura** | 30 procedimentos tRPC e 20 rotas Express — a totalidade dos handlers do servidor, lidos um a um. |
| **Resultado** | **5 achados** (2 médios, 3 baixos) e 1 observação informativa. Nenhum achado crítico ou alto. |

### Nota metodológica

As cinco categorias pedidas foram traduzidas para os mecanismos reais desta pilha antes da varredura. **(1) Banco sem tranca:** o projeto não usa Supabase nem RLS — o isolamento é feito por filtro manual sobre a coluna `openId` em cada consulta Drizzle, com o valor vindo sempre de `ctx.user.openId`; a auditoria verificou cada consulta de listagem, agregação e exportação. **(2) Permissão no navegador:** não há navegador — o cliente é React Native; cada gate de `userType` e de assinatura no aplicativo foi cruzado com o procedimento tRPC correspondente. **(3) IDOR:** todos os 50 handlers foram percorridos em busca de objeto localizado por identificador do cliente sem cruzamento com o dono. **(4) Chaves expostas:** varredura de código, configs, workflows, eas.json, scripts e histórico do Git, com atenção a valores padrão que viram segredo real. **(5) Inputs sem tratamento:** sem DOM, os equivalentes são a geração de HTML para `expo-print`, a landing HTML do servidor, os corpos de e-mail e as URLs entregues a `Linking.openURL` e ao WhatsApp. Somente achados confirmados no código foram reportados; onde a exploração depende de condições, elas estão descritas em cada ficha.

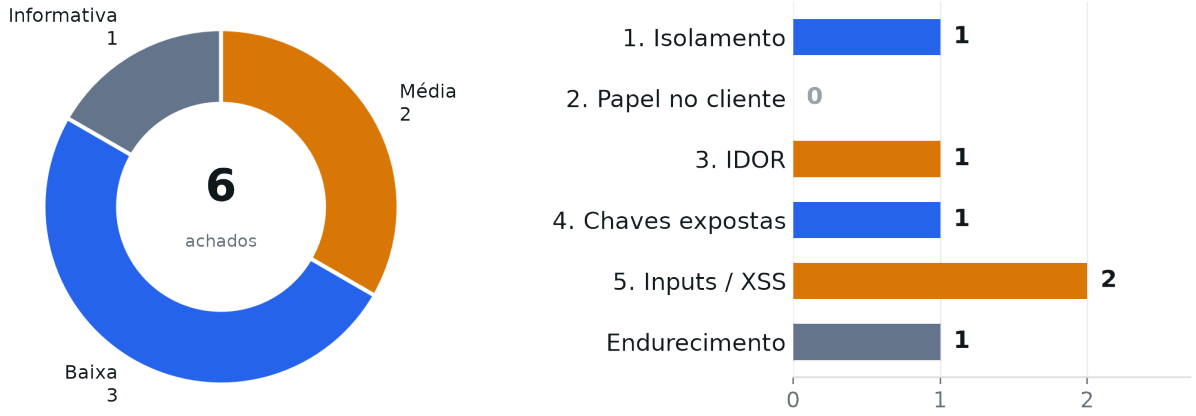
# Resumo executivo

| CRÍTICA | ALTA | MÉDIA | BAIXA | INFORMATIVA |
|---|---|---|---|---|
| 0 | 0 | 2 | 3 | 1 |

O backend do Vigora chega a esta auditoria em bom estado. O isolamento por conta é consistente e verificável em todos os 30 procedimentos tRPC, o acesso do cuidador aos dados da pessoa monitorada é derivado no servidor a partir do vínculo ativo, e as primitivas de autenticação — scrypt, códigos de uso único com teto de tentativas, convites com CSPRNG e resgate atômico — estão bem escolhidas e bem aplicadas. **Não foi encontrado nenhum achado crítico ou alto, nem qualquer segredo embutido no código ou no histórico do Git.**

Os cinco achados compartilham uma raiz: **exceções pontuais a padrões que o próprio projeto já aplica corretamente em outro lugar**. Um handler destrutivo sem verificação de posse, num backend onde todos os outros verificam (V-01); um campo que alimenta a mensagem de WhatsApp sem validação, enquanto o campo gêmeo é validado com `.url()` exatamente contra essa ameaça (V-02); um gerador de HTML sem escape ao lado de um irmão que documenta ter corrigido esse mesmo bug (V-03). Nenhum exige rearquitetura — todos são correções pequenas e localizadas.

## Achados por severidade




*À esquerda, distribuição por severidade. À direita, achados por categoria auditada — a cor de cada barra indica a severidade mais alta encontrada naquela categoria. A categoria 2 (permissão definida no cliente) fechou sem achados: todo gate de papel do aplicativo tem verificação equivalente no servidor.*

# Pontos fortes

O que foi verificado e está correto. Esta seção é também a prova de cobertura da auditoria: cada item cita o arquivo e as linhas inspecionadas.

> ✓ **Isolamento por conta em 100% dos 30 procedimentos tRPC**
> Os cinco routers (auth, userData, monitoring, link, push) foram lidos handler a handler. Todo acesso a dado de domínio deriva o escopo de **ctx.user.openId**, nunca de input do cliente. As consultas de listagem — getHistory (routers-monitoring.ts:296), getWarnings (310), getStatus (326), userData.get (routers.ts:243) e a exportação LGPD userData.export (296) — passam openId explicitamente, e as funções de banco correspondentes filtram por `eq(tabela.openId, openId)` (db-monitoring.ts:378, 450; db.ts:162; db-links.ts:145, 162).

**✓ Acesso do cuidador é derivado no servidor, jamais aceito do cliente**

**requireCaregiverLink** (routers-links.ts:93-102) resolve o vínculo ativo pelo openId de quem chama e devolve 403 se não houver. É o portão único de getMonitoredData (263) e getMonitoredAlerts (294) — nenhuma das duas aceita um identificador de pessoa monitorada por parâmetro. A revogação por qualquer das partes (revokeLink, 443-452) marca `status='revoked'` e todas as consultas de acesso filtram por `status='active'`, encerrando a leitura no ato.

**✓ Categoria 2 sem achados: todo gate de papel do cliente tem par no servidor**

Os gates de `userType` do aplicativo são espelhados no backend: createInvite exige monitored (routers-links.ts:110-115), redeemInvite exige caregiver (148-153), createShareInvite exige caregiver (326-331) e acceptInvite exige monitored (384-389). O guard de rota app/(caregiver-tabs)/_layout.tsx:21-34 é declaradamente defesa em profundidade. O `isPro` do RevenueCat controla apenas o selo visual (app/(tabs)/index.tsx:418 e tudo.tsx:330) — nenhuma capacidade de servidor depende dele. O **adminProcedure** existe (_core/trpc.ts:30-45) mas não é usado por nenhuma rota: não há superfície administrativa a proteger.

**✓ Nenhum segredo embutido no código nem no histórico do Git**

Varredura por padrões de chave/token/senha em código, configs, workflows e scripts não retornou credencial alguma — apenas segredos óbvios de teste em tests/*.test.ts. Os Client IDs do Google em google-auth.ts:30-35 e eas.json são identificadores públicos de OAuth, documentados como tal. O `.env` chegou a ser versionado (commit 6d65099, removido em a02346c), mas continha apenas marcadores (`<Android Client ID do Google Cloud Console>`) — não há vazamento no histórico. Chaveiros, credentials.json e google-service-account.json estão cobertos pelo .gitignore e nunca foram rastreados.

**✓ Escapamento de HTML correto nos dois geradores mais expostos**

A landing de convite renderizada pelo servidor valida o token contra `/^[A-Za-z0-9_-]{1,32}$/` antes de refleti-lo num href e passa todo o resto por **escapeHtml** (invite-landing.ts:95-125), servida sob CSP restritiva (_core/index.ts:94-97). O relatório de saúde em PDF roteia cada string do usuário por **esc()**, inclusive nos campos do cabeçalho, pré-escapados na atribuição (health-report-generator.ts:469-473). Nenhum e-mail transacional interpola input do usuário: os corpos em email-auth.ts:142-177 só recebem o código gerado no servidor.

**✓ Sessão, transporte e limites de taxa bem construídos**

JWT em **expo-secure-store** no nativo e cookie httpOnly na web, sem localStorage (lib/_core/auth.ts:106-157). CORS por allowlist que recusa preflight de origem desconhecida (_core/cors.ts:135-165), cabeçalhos de segurança completos incluindo HSTS e `frame-ancestors 'none'` (_core/security-headers.ts). Limite de taxa por IP não forjável, derivado de `req.ip` com `trust proxy 1` (_core/rate-limit.ts:213-224), somado a throttles por destino (db-auth.ts:245-260) e por conta em SOS, alertas, convites e resgates.

**✓ Senhas, códigos e convites com primitivas corretas**

scrypt N=16384 com sal por usuário e **timingSafeEqual** (email-auth.ts:59-99), hash descartável para equalizar o tempo e impedir enumeração (103, 334), códigos OTP via `randomInt` com TTL de 15 min, uso único e teto de 5 tentativas por linha (db-auth.ts:264-347). Convites usam CSPRNG — 6 caracteres com TTL de 10 min para o código falado e token de ~96 bits para o link (links-code.ts) — com resgate atômico `claim-before-act` que fecha o TOCTOU (db-links.ts:55-75).

**✓ Exclusão de conta transacional e sem dado de saúde em log**

**deleteAccountData** (db-account.ts:234-287) apaga as dez tabelas numa única transação e deixa a linha de `users` por último, invalidando toda sessão em aberto no commit. Busca por `console.*` contendo anamnese, métricas, tipo sanguíneo, medicamentos, contatos ou coordenadas não retornou nenhuma ocorrência; os números de telefone são mascarados antes de qualquer log (whatsapp.ts:214-216).

# Pontos fracos — os riscos centrais

**Um handler destrutivo sem verificação de posse**

**push.unregister** é a única exceção ao padrão de posse implícita que o resto do backend segue com rigor — e a exceção cai justamente sobre o canal de alerta em tempo real do dead man's switch, onde a falha é silenciosa (V-01).

**Validação de entrada desigual entre caminhos que terminam no mesmo lugar**

Dois campos alimentam a mesma mensagem de WhatsApp sob remetente confiável; um é validado com `.url()` e o outro aceita texto livre (V-02). O mesmo padrão se repete na geração de HTML: um módulo escapa tudo, o irmão não escapa nada (V-03). O risco central não é cada ponto isolado, e sim a ausência de uma regra uniforme aplicada a todo limite de confiança.

**Controles que dependem de configuração externa correta para valer**

A recusa de boot sem JWT_SECRET protege apenas quando `NODE_ENV` vale exatamente `production` (V-04). Uma trava de segurança não deveria depender de uma string de ambiente que vive fora do repositório.

**Escopo declarado mais estreito que o implementado**

O comentário de **sendEmergencyAlert** promete que todo destinatário é um contato cadastrado; a comparação por sufixo de 8 dígitos entrega algo consideravelmente mais amplo (V-05). Divergência entre documentação de segurança e código corrói a confiança em revisões futuras.

# Achados — visão consolidada

Ordenados por severidade. As fichas completas, com evidência de código, vêm logo em seguida.

| SEVERIDADE | ARQUIVO:LINHA | DESCRIÇÃO |
|---|---|---|
| MÉDIA | `server/routers-push.ts :91-96` | **V-01** — push.unregister apaga o token de push de qualquer conta, sem verificar posse |
| MÉDIA | `server/routers-monitoring.ts :203 (entrada) → server/monitoring-job.ts:470-476 (uso)` | **V-02** — lastLocation entra sem validação na mensagem de WhatsApp enviada aos contatos de emergência |
| BAIXA | `lib/pdf-utils-v2.ts :89, 93, 111, 115, 119, 127, 131, 135` | **V-03** — PDF da ficha médica monta HTML sem escapar nenhum campo do usuário |
| BAIXA | `server/_core/env.ts :9 e 99-110; server/_core/sdk.ts:50-53` | **V-04** — JWT_SECRET tem fallback para string vazia; a trava de inicialização depende só de NODE_ENV |
| BAIXA | `server/routers.ts :33-44 (uso em 417-426)` | **V-05** — Lista de destinatários do alerta valida só os 8 últimos dígitos do telefone |
| INFORMATIVA | `server/_core/index.ts :46` | **V-06** — Parser de formulário registrado numa API exclusivamente JSON |

# Achados detalhados

MÉDIA

## V-01 — push.unregister apaga o token de push de qualquer conta, sem verificar posse

**Categoria 3. IDOR (objeto por ID)** · `server/routers-push.ts:91-96`

### Evidência

```
unregister: protectedProcedure
  .input(z.object({ token: z.string().min(1).max(255) }))
  .mutation(async ({ input }) => {
    await deletePushToken(input.token);   // <- sem ctx.user.openId
    return { success: true } as const;
  }),
```

### Por que é explorável

É o único handler do backend que localiza uma linha por identificador vindo do cliente e a apaga sem cruzar com **ctx.user.openId**. O **protectedProcedure** garante apenas que existe uma sessão válida — não que o token pertence a quem chama. A função de banco **deletePushToken** (server/db-push.ts:42-46) filtra só por `eq(pushTokens.token, token)`. O comportamento é deliberado e está travado por teste (tests/push-unregister.test.ts:80-93, "apaga a linha mesmo quando ela pertence a outra conta"), o que significa que uma correção precisa preservar o caso de uso legítimo — aparelho que registrou como cuidador e depois entrou como monitorado.

### Impacto

Apagar o token do cuidador desarma silenciosamente o canal de alerta em tempo real do dead man's switch: **pushMissedAlarmToCaregivers**, **sosAlertCaregivers** e os Passos 3 e 4 do monitoring-job resolvem zero tokens e saem sem notificar. O cuidador só descobriria abrindo a tela de Alertas. É o cenário de falha mais grave do produto (alarme não entregue) provocado por uma chamada autenticada de terceiro.

### Condições de explorabilidade

Exige conhecer o valor do Expo push token da vítima. Nenhuma rota do servidor devolve tokens ao cliente (getPushTokensForOpenIds é usado só internamente), então não há enumeração remota: o vetor realista é vazamento por log, aparelho compartilhado, backup ou comprometimento do dispositivo.

### Correção sugerida

Apagar por `(token, openId)` quando a linha pertence a quem chama e, para o caso de troca de conta no mesmo aparelho, exigir prova de posse do dispositivo — por exemplo aceitar o par (token, deviceId) ou deixar a limpeza da linha órfã para o **upsertPushToken** do próximo registro, que já reatribui o openId da linha. No mínimo, aplicar limite de taxa por chamador e registrar log da remoção cruzada.

MÉDIA

## V-02 — lastLocation entra sem validação na mensagem de WhatsApp enviada aos contatos de emergência

**Categoria 5. Inputs sem tratamento (XSS)** · `server/routers-monitoring.ts:203 (entrada)` → `server/monitoring-job.ts:470-476 (uso)`

### Evidência

```
// routers-monitoring.ts:203 — heartbeat aceita texto livre
lastLocation: z.string().optional(),

// monitoring-job.ts:470-476 — o texto vira URL dentro da mensagem
const liveness = await getAccountLiveness(account.openId);
if (liveness?.lastLocation) {
  const [lat, lng] = liveness.lastLocation.split(",");
  if (lat && lng) {
    locationUrl = `https://maps.google.com/?q=${lat},${lng}`;
  }
}
// buildWarningMessage (monitoring-job.ts:228-231)
message += `\n\n■ Última localização registrada:\n${locationUrl}`;
```

### Por que é explorável

**lastLocation** é aceito como `z.string()` sem formato nem tamanho, gravado direto em **account_liveness.lastLocation** (server/db-monitoring.ts:41-43) e depois interpolado numa URL que compõe o corpo da mensagem enviada pelo número do WhatsApp Business do Vigora, com `preview_url: true` (server/whatsapp.ts:231). O `split(",")` não valida que as partes sejam coordenadas: qualquer texto com uma vírgula produz um `locationUrl` arbitrário. O próprio código reconhece esse risco no caminho gêmeo — em server/routers.ts:385 o campo `locationUrl` é validado com `.url()` justamente para que "texto arbitrário/phishing não seja injetado no corpo templado do WhatsApp sob o remetente confiável Vigora". A mesma defesa não existe nesta rota.

### Impacto

Injeção de conteúdo em mensagem enviada sob remetente verificado a familiares idosos, num momento de pânico ("alerta sério"). Permite phishing com alta taxa de sucesso e, se abusado, coloca em risco a reputação do número do WhatsApp Business — cujo banimento pela Meta derrubaria a escalação de *todos* os usuários.

### Condições de explorabilidade

Exige controle de uma sessão válida da conta monitorada (o próprio aparelho, uma sessão roubada ou um cliente modificado) — a mensagem envenenada vai para os contatos daquela conta. A coluna é `varchar(64)`, o que limita o payload; conforme o `sql_mode` do MySQL o valor é truncado (silencioso) ou rejeitado.

### Correção sugerida

Validar o formato no limite tRPC — por exemplo `z.string().regex(/^-?\d{1,3}(\.\d+)?,-?\d{1,3}(\.\d+)?$/)` — e, em monitoring-job.ts, converter cada parte com `Number()` e só montar a URL se ambas forem finitas e dentro das faixas de latitude/longitude, aplicando `encodeURIComponent` na concatenação.

BAIXA

## V-03 — PDF da ficha médica monta HTML sem escapar nenhum campo do usuário

**Categoria 5. Inputs sem tratamento (XSS)** · `lib/pdf-utils-v2.ts:89, 93, 111, 115, 119, 127, 131, 135`

### Evidência

```
<div class="field-value">${anamnesis.fullName || 'Não informado'}</div>
<div class="field-value">${formatDate(anamnesis.birthDate)}</div>
<div class="field-value">${anamnesis.allergies || 'Nenhuma informada'}</div>
<div class="field-value">${anamnesis.medications || 'Nenhum informado'}</div>
<div class="field-value">${anamnesis.diseases || 'Nenhuma informada'}</div>
<div class="field-value">${anamnesis.susNumber || 'Não informado'}</div>
<div class="field-value">${anamnesis.healthPlanProvider || 'Não informado'}</div>
<div class="field-value">${anamnesis.healthPlanNumber || 'Não informado'}</div>
```

### Por que é explorável

Oito campos de texto livre da anamnese são concatenados crus no HTML que **expo-print** renderiza (`Print.printToFileAsync`, lib/pdf-utils-v2.ts:158) — no Android isso é um WebView de impressão. O módulo irmão **lib/health-report-generator.ts** já corrigiu exatamente esta classe de bug: ele define `esc()` (linhas 28-36) e o cabeçalho do arquivo documenta que "a versão anterior concatenava input cru do usuário no HTML, permitindo injeção trivial de HTML/script". O pdf-utils-v2 nunca recebeu esse tratamento — não há nenhuma chamada de escape no arquivo.

### Impacto

Injeção de HTML/CSS num documento médico compartilhado externamente (`Sharing.shareAsync`, linha 170): o conteúdo do PDF entregue ao profissional de saúde pode ser adulterado — campos ocultados, valores sobrepostos, texto forjado. No WebView de impressão do Android há ainda execução de script no contexto da renderização.

### Condições de explorabilidade

A anamnese renderizada é a da própria conta (app/(tabs)/anamnesis.tsx:126 passa o formulário local), então é autoinjeção: exige que o dado malicioso já esteja no estado da conta. Nenhuma tela de cuidador chama este gerador, portanto não há caminho entre contas. Vale como defeito de consistência e defesa em profundidade, não como tomada de conta.

### Correção sugerida

Importar e aplicar o `esc()` já exportado por lib/health-report-generator.ts em todos os oito pontos de interpolação (e nas partes de `formatDate`). Um teste que renderize uma anamnese com `<script>` e afirme a ausência da tag no HTML de saída trava a regressão.

BAIXA

## V-04 — JWT_SECRET tem fallback para string vazia; a trava de inicialização depende só de NODE_ENV

**Categoria 4. Chaves expostas (hardcode)** · `server/_core/env.ts:9 e 99-110; server/_core/sdk.ts:50-53`

### Evidência

```
// env.ts:9
cookieSecret: process.env.JWT_SECRET ?? "",

// sdk.ts:50-53 — assina E verifica com a chave vazia se a env faltar
function getSessionSecret() {
  const secret = process.env.JWT_SECRET ?? "";
  return new TextEncoder().encode(secret);
}

// env.ts:99-110 — a recusa de boot só vale quando NODE_ENV === "production"
const isProduction = env.NODE_ENV === "production";
if (!isProduction) {
  if (secret.length === 0) { console.warn(...); }
  return;                        // <- segue com HMAC de chave vazia
}
```

### Por que é explorável

O valor padrão vazio é exatamente o "default que vira segredo real se não for sobrescrito": com ele, os JWTs de sessão são assinados *e* verificados com uma chave HMAC vazia, permitindo forjar token para qualquer **openId**. A validação de startup **assertRequiredSecrets** existe e está correta — mas decide pela string `NODE_ENV` em vez de decidir pela presença do segredo. Qualquer desvio (`"Production"`, `"prod"`, variável ausente porque um Start Command customizado no painel do Railway substituiu o script `start` — que é quem define `NODE_ENV=production`, package.json:10) faz o servidor subir em modo permissivo, com apenas um aviso no log.

### Impacto

No cenário em que a condição se materializa, o impacto é total: forja de sessão para qualquer conta, com acesso a métricas de saúde, anamnese, contatos de emergência e localização. Nada mais no sistema barra um JWT bem-formado.

### Condições de explorabilidade

Não explorável na configuração atual verificada: o script `start` define NODE_ENV=production e não há Dockerfile, Procfile, railway.json ou nixpacks.toml no repositório que contradiga isso. O risco é de configuração — a trava não é auto-suficiente. Também não foi encontrado nenhum segredo real embutido no código, nos workflows, no eas.json ou no histórico do Git (ver Pontos Fortes).

### Correção sugerida

Inverter a decisão: exigir o segredo sempre que ele for usado, e permitir o modo permissivo apenas mediante uma opção explícita de desenvolvimento (por exemplo `ALLOW_INSECURE_DEV_SECRET=1`), nunca por inferência de NODE_ENV. Alternativa mínima e barata: fazer `getSessionSecret()` lançar quando o segredo é vazio, de modo que a falha ocorra na primeira operação de sessão em vez de aceitar tokens forjados. Estender a checagem a DATABASE_URL fecha a mesma lacuna no banco.

## BAIXA V-05 — Lista de destinatários do alerta valida só os 8 últimos dígitos do telefone

**Categoria 1. Banco sem tranca (isolamento por conta)** · `server/routers.ts:33-44 (uso em 417-426)`

### Evidência

```
function isAllowedRecipient(claimed, stored): boolean {
  const claimedDigits = normalizeDigits(claimed.phone);
  if (claimedDigits.length < 8) return false;
  const claimedTail = claimedDigits.slice(-8);      // <- só o sufixo
  return stored.some((c) => {
    const tail = normalizeDigits(c.phone).slice(-8);
    return tail.length >= 8 && tail === claimedTail;
  });
}
```

### Por que é explorável

O controle existe para garantir que **whatsapp.sendEmergencyAlert** não tenha "destinos arbitrários" (comentário em server/routers.ts:365-368). Comparar apenas os 8 dígitos finais descarta o DDI, o DDD e o nono dígito: um contato salvo em `(11) 9 8888-7777` autoriza o envio para `(51) 8888-7777`, `(21) 9 8888-7777` e todas as demais variantes de DDD/prefixo — cerca de uma centena de números reais distintos por contato cadastrado, em vez de um. O escopo efetivo do controle é bem mais largo que o documentado.

### Impacto

Envio de mensagem sob o remetente confiável do Vigora para números que o titular nunca cadastrou. O teto de abuso é contido pelo limite de taxa por conta (5 mensagens / 60 s, server/routers.ts:51-66) e pelo fato de o alvo ter que colidir no sufixo, mas a garantia de "apenas contatos do titular" não se sustenta como escrita.

### Condições de explorabilidade

Requer sessão autenticada. O ganho real para um atacante é limitado, porque a lista de contatos é gravada pelo próprio titular via **userData.put** — quem controla a conta já pode cadastrar o número que quiser. O achado é relevante como divergência entre a propriedade de segurança declarada e a implementada, e como enfraquecimento do controle diante de um cliente comprometido.

### Correção sugerida

Comparar o número normalizado por completo (dígitos com DDI, via a mesma **normalizeBrPhone** de server/phone-auth.ts:41-59) em vez do sufixo. Se a tolerância a variações de formato precisar ser mantida, normalizar os dois lados para E.164 antes de comparar, tratando explicitamente o nono dígito, em vez de truncar.

---

## INFORMATIVA V-06 — Parser de formulário registrado numa API exclusivamente JSON

**Categoria Endurecimento (informativo)** · `server/_core/index.ts:46`

### Evidência

```
app.use(express.json({ limit: "1mb" }));
app.use(express.urlencoded({ limit: "1mb", extended: true }));  // <- sem uso
```

### Por que é explorável

Nenhuma rota consome corpo `application/x-www-form-urlencoded`: as dez rotas de `/api/auth` fazem `safeParse(req.body)` sobre JSON e o restante passa pelo adaptador tRPC. O parser aumenta a superfície para requisições "simples" (sem preflight CORS), que são as únicas que atravessariam a política de origem com o cookie de sessão `SameSite=none` anexado.

### Impacto

Baixo. **Verificado empiricamente** com uma prova de conceito local (express + @trpc/server nas versões do projeto): o tRPC v11 responde `415 Unsupported Media Type` tanto para `application/x-www-form-urlencoded` quanto para `text/plain`, e a mutação não executa. Não há CSRF contra os procedimentos tRPC. O resíduo se limita a rotas express simples sem efeito sensível — `POST /api/auth/logout`, que apenas limparia o cookie da vítima.

### Condições de explorabilidade

Nenhuma exploração confirmada. Registrado como redução de superfície.

### Correção sugerida

Remover a linha 46. A API é JSON de ponta a ponta e nada depende do parser.

# Recomendações priorizadas

P1 é o que deve entrar antes do lançamento; P2 e P3 no ciclo seguinte; P4 e P5 são endurecimento e processo.

| # | AÇÃO | DETALHE |
|---|---|---|
| P1 | **Fechar V-01 e V-02 antes do lançamento** | São os dois achados que tocam o diferencial do produto. V-01 permite desarmar o alerta ao cuidador com uma chamada autenticada; V-02 permite injetar conteúdo na mensagem que a família recebe sob o remetente Vigora. Ambos têm correção pequena e localizada, e ambos merecem teste de regressão — o padrão de tests/push-unregister.test.ts serve de molde, invertendo a asserção do terceiro caso. |
| P2 | **Uniformizar validação e escapamento nos limites de confiança** | Aplicar `esc()` nos oito pontos de lib/pdf-utils-v2.ts (V-03) e trocar a comparação por sufixo por telefone normalizado completo em isAllowedRecipient (V-05). Convém registrar a regra em docs/claude/padroes-e-testes.md: todo dado que sai para HTML, URL ou mensagem passa por escape/validação explícita, sem exceção por módulo. |
| P3 | **Tornar a trava de segredo auto-suficiente** | Fazer a exigência do JWT_SECRET depender da presença do segredo e não de NODE_ENV, com escape explícito para desenvolvimento (V-04), e estender a verificação a DATABASE_URL. Complementar com um teste que afirme que a função de assinatura recusa segredo vazio. |
| P4 | **Reduzir superfície e fechar lacunas de consistência** | Remover `express.urlencoded` (V-06). Como itens menores observados durante a varredura, sem achado associado: **link.getInviteInfo** (routers-links.ts:360) é o único procedimento do router de vínculos sem limite de taxa próprio — inofensivo hoje, porque o token tem ~96 bits, mas fora do padrão dos irmãos; e **consumeInviteByCode** devolve `true` quando não há banco (db-links.ts:61), um padrão fail-open hoje inalcançável (a autenticação falha antes) que convém inverter. |
| P5 | **Cobrir o processo, não só o código** | Duas observações de arquitetura registradas na leitura: a denylist de tokens revogados e todos os limitadores de taxa são mapas em memória por processo (sdk.ts:21, rate-limit.ts:236, routers-links.ts:50-63) — corretos para uma instância, mas silenciosamente ineficazes se o deploy do Railway escalar horizontalmente; e o /api/health já expõe a saúde do job do dead man's switch, mas o incidente de 27 h registrado no repositório mostra que monitoramento externo ativo é o que fecha esse laço. |

# Issues para o GitHub

Texto completo de cada issue, em Markdown, pronto para copiar e colar. Cada bloco esta delimitado por `--- ISSUE n ---` e `--- FIM ISSUE n ---`. Achados triviais relacionados foram mantidos juntos quando compartilham o mesmo tema, para não gerar ruído de issues.

**--- ISSUE 1 (V-01) ---**

```
# [Segurança] push.unregister apaga token de push sem verificar posse

**Labels:** `security`, `severity:media`, `backend`

## Problema

`push.unregister` recebe um token de push do cliente e apaga a linha
correspondente sem cruzar com `ctx.user.openId`. O `protectedProcedure`
garante apenas que existe uma sessão valida - nao que o token pertence a
quem chama. E o unico handler do backend que localiza um objeto por
identificador vindo do cliente e o remove sem verificacao de dono.

O comportamento e deliberado (resolve o caso do aparelho que registrou como
cuidador e depois entrou como monitorado) e esta travado por teste, entao a
correcao precisa preservar esse caso de uso.

## Evidencia

server/routers-push.ts:91-96

    unregister: protectedProcedure
      .input(z.object({ token: z.string().min(1).max(255) }))
      .mutation(async ({ input }) => {
        await deletePushToken(input.token);   // sem ctx.user.openId
        return { success: true } as const;
      }),

server/db-push.ts:42-46

    export async function deletePushToken(token: string): Promise<void> {
      const db = await getDb();
      if (!db) return;
      await db.delete(pushTokens).where(eq(pushTokens.token, token));
    }

tests/push-unregister.test.ts:80-93 documenta e trava o comportamento atual
("apaga a linha mesmo quando ela pertence a outra conta").

## Impacto

Apagar o token do cuidador desarma silenciosamente o canal de alerta em
tempo real do dead man's switch. `pushMissedAlarmToCaregivers`,
`sosAlertCaregivers` e os Passos 3 e 4 do monitoring-job passam a resolver
zero tokens e saem sem notificar ninguem. O cuidador so perceberia abrindo a
tela de Alertas manualmente.

Explorabilidade: exige conhecer o valor do Expo push token da vitima.
Nenhuma rota devolve tokens ao cliente, entao nao ha enumeracao remota - o
vetor realista e vazamento por log, aparelho compartilhado ou backup.

## Sugestao de correcao

Apagar por `(token, openId)` quando a linha pertence a quem chama. Para o
caso legitimo de troca de conta no mesmo aparelho, exigir prova de posse do
dispositivo (aceitar o par `token` + `deviceId`), ou deixar a limpeza da
linha orfa para o `upsertPushToken` do proximo registro, que ja reatribui o
openId da linha via onDuplicateKeyUpdate.

No minimo: limite de taxa por chamador e log da remocao cruzada.

## Criterios de aceite

- [ ] `push.unregister` nao apaga linha de outra conta sem prova de posse do
      aparelho
- [ ] O caso "aparelho trocou de conta" continua funcionando (o token para de
```

```
      receber push apos o logout)
- [ ] tests/push-unregister.test.ts atualizado: o caso "apaga a linha mesmo
      quando ela pertence a outra conta" passa a exigir a prova de posse
- [ ] Novo teste: chamador autenticado sem prova de posse nao consegue apagar
      token de terceiro
- [ ] `pnpm test` e `pnpm check` passam
```

**--- FIM ISSUE 1 ---**

**--- ISSUE 2 (V-02) ---**

```
# [Segurança] lastLocation sem validacao entra na mensagem de WhatsApp

**Labels:** `security`, `severity:media`, `backend`, `lgpd`

## Problema

`monitoring.heartbeat` aceita `lastLocation` como string livre, sem formato
nem tamanho. O valor e gravado em `account_liveness.lastLocation` e depois
interpolado numa URL que compoe o corpo da mensagem enviada pelo numero do
WhatsApp Business do Vigora, com `preview_url: true`.

O `split(",")` nao valida que as partes sejam coordenadas: qualquer texto
contendo uma virgula produz um `locationUrl` arbitrario dentro de uma
mensagem que sai sob remetente confiavel.

O caminho gemeo ja se defende disso: em `whatsapp.sendEmergencyAlert` o
campo `locationUrl` e validado com `.url()`, e o comentario diz
explicitamente que e "para que texto arbitrario/phishing nao seja injetado no
corpo templado do WhatsApp sob o remetente confiavel Vigora". A mesma defesa
nao existe nesta rota.

## Evidencia

server/routers-monitoring.ts:203 (entrada)

    lastLocation: z.string().optional(),

server/db-monitoring.ts:41-43 (persistencia)

    const locationFields = meta?.lastLocation
      ? { lastLocation: meta.lastLocation, lastLocationAt: now }
      : {};

server/monitoring-job.ts:470-476 (uso)

    const liveness = await getAccountLiveness(account.openId);
    if (liveness?.lastLocation) {
      const [lat, lng] = liveness.lastLocation.split(",");
      if (lat && lng) {
        locationUrl = `https://maps.google.com/?q=${lat},${lng}`;
      }
    }

server/monitoring-job.ts:228-231 (envio)

    if (locationUrl) {
      message += `\n\nUltima localizacao registrada:\n${locationUrl}`;
    }

Compare com server/routers.ts:383-385, o caminho ja protegido:

    // .url() so arbitrary text/phishing can't be injected into the
    // templated WhatsApp body under the trusted "Vigora" sender.
    locationUrl: z.string().url().max(500).optional(),

## Impacto

Injecao de conteudo em mensagem enviada sob remetente verificado a
familiares idosos, num momento de panico ("alerta serio"). Alta taxa de
sucesso para phishing. Se abusado em escala, coloca em risco a reputacao do
numero do WhatsApp Business - cujo banimento pela Meta derrubaria a
escalacao de todos os usuarios.

Explorabilidade: exige controle de uma sessao valida da conta monitorada
(aparelho, sessao roubada ou cliente modificado). A coluna e varchar(64), o
```

```
que limita o payload; conforme o sql_mode do MySQL o valor e truncado ou
rejeitado.

## Sugestao de correcao

Validar no limite tRPC:

    lastLocation: z
      .string()
      .regex(/^-?\d{1,3}(\.\d+)?,-?\d{1,3}(\.\d+)?$/)
      .max(64)
      .optional(),

E, em monitoring-job.ts, converter cada parte com `Number()`, montar a URL
apenas se ambas forem finitas e dentro das faixas de latitude/longitude, e
aplicar `encodeURIComponent` na concatenacao.

## Criterios de aceite

- [ ] `monitoring.heartbeat` rejeita `lastLocation` que nao seja um par
      "lat,lng" numerico
- [ ] `monitoring-job` nao monta `locationUrl` a partir de valor invalido
      (defesa em profundidade para linhas ja gravadas no banco)
- [ ] Teste: heartbeat com `lastLocation` contendo texto/URL e recusado
- [ ] Teste: valor invalido ja persistido nao aparece na mensagem gerada por
      `buildWarningMessage`
- [ ] `pnpm test` e `pnpm check` passam
```

**--- FIM ISSUE 2 ---**

**--- ISSUE 3 (V-03) ---**

```
# [Segurança] PDF da ficha medica monta HTML sem escapar campos do usuario

**Labels:** `security`, `severity:baixa`, `frontend`

## Problema

`lib/pdf-utils-v2.ts` concatena oito campos de texto livre da anamnese
diretamente no HTML que o `expo-print` renderiza - no Android, um WebView de
impressao. Nao ha nenhuma chamada de escape no arquivo.

O modulo irmao `lib/health-report-generator.ts` ja corrigiu exatamente esta
classe de bug: define `esc()` e o cabecalho do arquivo documenta que "a
versao anterior concatenava input cru do usuario no HTML, permitindo injecao
trivial de HTML/script". O pdf-utils-v2 nunca recebeu esse tratamento.

## Evidencia

lib/pdf-utils-v2.ts - linhas 89, 93, 111, 115, 119, 127, 131, 135

    <div class="field-value">${anamnesis.fullName || 'Nao informado'}</div>
    <div class="field-value">${formatDate(anamnesis.birthDate)}</div>
    <div class="field-value">${anamnesis.allergies || 'Nenhuma informada'}</div>
    <div class="field-value">${anamnesis.medications || 'Nenhum informado'}</div>
    <div class="field-value">${anamnesis.diseases || 'Nenhuma informada'}</div>
    <div class="field-value">${anamnesis.susNumber || 'Nao informado'}</div>
    <div class="field-value">${anamnesis.healthPlanProvider || 'Nao informado'}</div>
    <div class="field-value">${anamnesis.healthPlanNumber || 'Nao informado'}</div>

lib/pdf-utils-v2.ts:158 - o HTML vai para o renderizador

    const { uri } = await Print.printToFileAsync({ html: htmlContent, ... });

lib/health-report-generator.ts:28-36 - o escape que ja existe no projeto

    export function esc(value: unknown): string {
      if (value === null || value === undefined) return '';
      return String(value)
        .replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;').replace(/'/g, '&#39;');
    }

## Impacto

Injecao de HTML/CSS num documento medico compartilhado externamente via
```

```
`Sharing.shareAsync`: o PDF entregue ao profissional de saude pode ter campos
ocultados, valores sobrepostos ou texto forjado. No WebView de impressao do
Android ha ainda execucao de script no contexto da renderizacao.

Explorabilidade: a anamnese renderizada e a da propria conta
(app/(tabs)/anamnesis.tsx:126 passa o formulario local), entao e
autoinjecao. Nenhuma tela de cuidador chama este gerador - nao ha caminho
entre contas. Vale como defeito de consistencia e defesa em profundidade.

## Sugestao de correcao

Importar `esc` de `@/lib/health-report-generator` e aplicar nos oito pontos
de interpolacao, incluindo as partes usadas por `formatDate`.

## Criterios de aceite

- [ ] Os oito campos passam por `esc()` antes de entrar no HTML
- [ ] `formatDate` nao devolve fragmento nao escapado
- [ ] Teste: anamnese com `<script>alert(1)</script>` em `allergies` gera HTML
      sem a tag `<script>`
- [ ] O PDF continua sendo gerado e compartilhado normalmente
- [ ] `pnpm test` e `pnpm check` passam
```

**--- FIM ISSUE 3 ---**

**--- ISSUE 4 (V-04) ---**

```
# [Segurança] JWT_SECRET com fallback vazio e trava presa a NODE_ENV

**Labels:** `security`, `severity:baixa`, `backend`, `infra`

## Problema

`JWT_SECRET` cai para string vazia quando a variavel nao esta definida. Com
ela, os JWTs de sessao sao assinados E verificados com uma chave HMAC vazia -
forja de token para qualquer `openId`.

A validacao de startup `assertRequiredSecrets` existe e esta bem escrita, mas
decide pela string `NODE_ENV` em vez de decidir pela presenca do segredo.
Qualquer desvio ("Production", "prod", ou variavel ausente porque um Start
Command customizado no painel do Railway substituiu o script `start` - que e
quem define NODE_ENV=production) faz o servidor subir em modo permissivo com
apenas um aviso no log.

Nao ha exploracao confirmada na configuracao atual: o script `start` define
NODE_ENV=production e nao ha Dockerfile, Procfile, railway.json ou
nixpacks.toml no repositorio que contradiga isso. O problema e que a trava
nao e auto-suficiente.

## Evidencia

server/_core/env.ts:9

    cookieSecret: process.env.JWT_SECRET ?? "",

server/_core/sdk.ts:50-53

    function getSessionSecret() {
      const secret = process.env.JWT_SECRET ?? "";
      return new TextEncoder().encode(secret);
    }

server/_core/env.ts:99-110

    const isProduction = env.NODE_ENV === "production";
    const secret = env.JWT_SECRET ?? "";
    if (!isProduction) {
      if (secret.length === 0) { console.warn(...); }
      return;                    // segue com HMAC de chave vazia
    }

package.json:10 - a unica coisa que garante NODE_ENV em producao

    "start": "NODE_ENV=production node dist/index.js",

## Impacto
```

```
No cenario em que a condicao se materializa o impacto e total: forja de
sessao para qualquer conta, com acesso a metricas de saude, anamnese,
contatos de emergencia e localizacao. Nada mais no sistema barra um JWT
bem-formado.

## Sugestao de correcao

Inverter a decisao: exigir o segredo sempre que ele for usado e permitir o
modo permissivo apenas mediante opcao explicita de desenvolvimento
(`ALLOW_INSECURE_DEV_SECRET=1`), nunca por inferencia de NODE_ENV.

Alternativa minima: fazer `getSessionSecret()` lancar quando o segredo e
vazio, de modo que a falha ocorra na primeira operacao de sessao em vez de
aceitar tokens forjados.

Estender a checagem a `DATABASE_URL` fecha a mesma lacuna no banco.

## Criterios de aceite

- [ ] Servidor recusa iniciar sem `JWT_SECRET`, independente de `NODE_ENV`
- [ ] Modo permissivo de desenvolvimento so e ativado por variavel explicita
- [ ] `getSessionSecret()` lanca em vez de devolver chave vazia
- [ ] `assertRequiredSecrets` tambem valida `DATABASE_URL`
- [ ] Teste: com JWT_SECRET vazio e NODE_ENV="dev", o boot falha (ou a
      assinatura lanca)
- [ ] `pnpm test` e `pnpm check` passam
```

**--- FIM ISSUE 4 ---**

**--- ISSUE 5 (V-05) ---**

```
# [Segurança] Allowlist de destinatarios compara so 8 digitos finais

**Labels:** `security`, `severity:baixa`, `backend`

## Problema

`isAllowedRecipient` existe para garantir que `whatsapp.sendEmergencyAlert`
nao tenha "destinos arbitrarios". A comparacao usa apenas os 8 digitos
finais, descartando DDI, DDD e o nono digito.

Um contato salvo como `(11) 9 8888-7777` autoriza envio para
`(51) 8888-7777`, `(21) 9 8888-7777` e todas as demais variantes de
DDD/prefixo - cerca de uma centena de numeros reais distintos por contato
cadastrado, em vez de um. O escopo efetivo do controle e bem mais largo que o
documentado no proprio comentario da rota.

## Evidencia

server/routers.ts:33-44

    function isAllowedRecipient(claimed, stored): boolean {
      const claimedDigits = normalizeDigits(claimed.phone);
      if (claimedDigits.length < 8) return false;
      const claimedTail = claimedDigits.slice(-8);      // so o sufixo
      return stored.some((c) => {
        const tail = normalizeDigits(c.phone).slice(-8);
        return tail.length >= 8 && tail === claimedTail;
      });
    }

server/routers.ts:365-368 - a garantia declarada

    // SECURITY: Requires authentication and verifies that:
    //   1. Every phone number in `contacts` matches a stored emergency
    //      contact for ctx.user's account (no arbitrary destinations)

## Impacto

Envio de mensagem sob o remetente confiavel do Vigora para numeros que o
titular nunca cadastrou. O teto de abuso e contido pelo limite de taxa por
conta (5 mensagens / 60s) e pela necessidade de colisao no sufixo, mas a
garantia de "apenas contatos do titular" nao se sustenta como escrita.

Explorabilidade: requer sessao autenticada. O ganho real para um atacante e
```

```
limitado, porque a lista de contatos e gravada pelo proprio titular via
`userData.put` - quem controla a conta ja pode cadastrar o numero que quiser.
O achado importa como divergencia entre a propriedade de seguranca declarada
e a implementada, e como enfraquecimento do controle diante de um cliente
comprometido.

## Sugestao de correcao

Comparar o numero normalizado por completo, reaproveitando `normalizeBrPhone`
de server/phone-auth.ts:41-59 nos dois lados antes de comparar. Se a
tolerancia a variacoes de formato precisar ser mantida, tratar
explicitamente o nono digito em vez de truncar.

## Criterios de aceite

- [ ] `isAllowedRecipient` compara telefone normalizado completo (com DDI)
- [ ] Teste: contato salvo em DDD 11 nao autoriza envio para o mesmo sufixo em
      DDD 51
- [ ] Teste: variacoes legitimas de formato do MESMO numero continuam
      autorizadas ("(11) 99999-9999", "5511999999999", "+55 11 9 9999-9999")
- [ ] O comentario de SECURITY da rota descreve o comportamento real
- [ ] `pnpm test` e `pnpm check` passam
```

**--- FIM ISSUE 5 ---**

**--- ISSUE 6 (V-06) ---**

```
# [Segurança] Remover express.urlencoded de uma API exclusivamente JSON

**Labels:** `security`, `severity:informativa`, `backend`, `hardening`

## Problema

O servidor registra um parser de `application/x-www-form-urlencoded` que
nenhuma rota consome: as dez rotas de `/api/auth` fazem
`safeParse(req.body)` sobre JSON e o restante passa pelo adaptador tRPC.

O parser aumenta a superficie para requisicoes "simples" (sem preflight
CORS), que sao as unicas que atravessariam a politica de origem com o cookie
de sessao `SameSite=none` anexado.

## Evidencia

server/_core/index.ts:45-46

    app.use(express.json({ limit: "1mb" }));
    app.use(express.urlencoded({ limit: "1mb", extended: true }));  // sem uso

## Impacto

Baixo, e verificado empiricamente. Uma prova de conceito local (express +
@trpc/server nas versoes do projeto) confirmou que o tRPC v11 responde
`415 Unsupported Media Type` tanto para `application/x-www-form-urlencoded`
quanto para `text/plain`, e a mutacao nao executa:

    urlencoded  -> 415 | mutation executou? false
    text/plain  -> 415 | mutation executou? false

Nao ha CSRF contra os procedimentos tRPC. O residuo se limita a rotas express
simples sem efeito sensivel - `POST /api/auth/logout`, que apenas limparia o
cookie da vitima.

Registrado como reducao de superficie, nao como vulnerabilidade explorada.

## Sugestao de correcao

Remover a linha 46. A API e JSON de ponta a ponta.

## Criterios de aceite

- [ ] `express.urlencoded` removido de server/_core/index.ts
- [ ] Todas as rotas de `/api/auth` continuam funcionando (JSON)
- [ ] `pnpm test` e `pnpm check` passam
```

**--- FIM ISSUE 6 ---**